

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Nguyễn Đông Quân

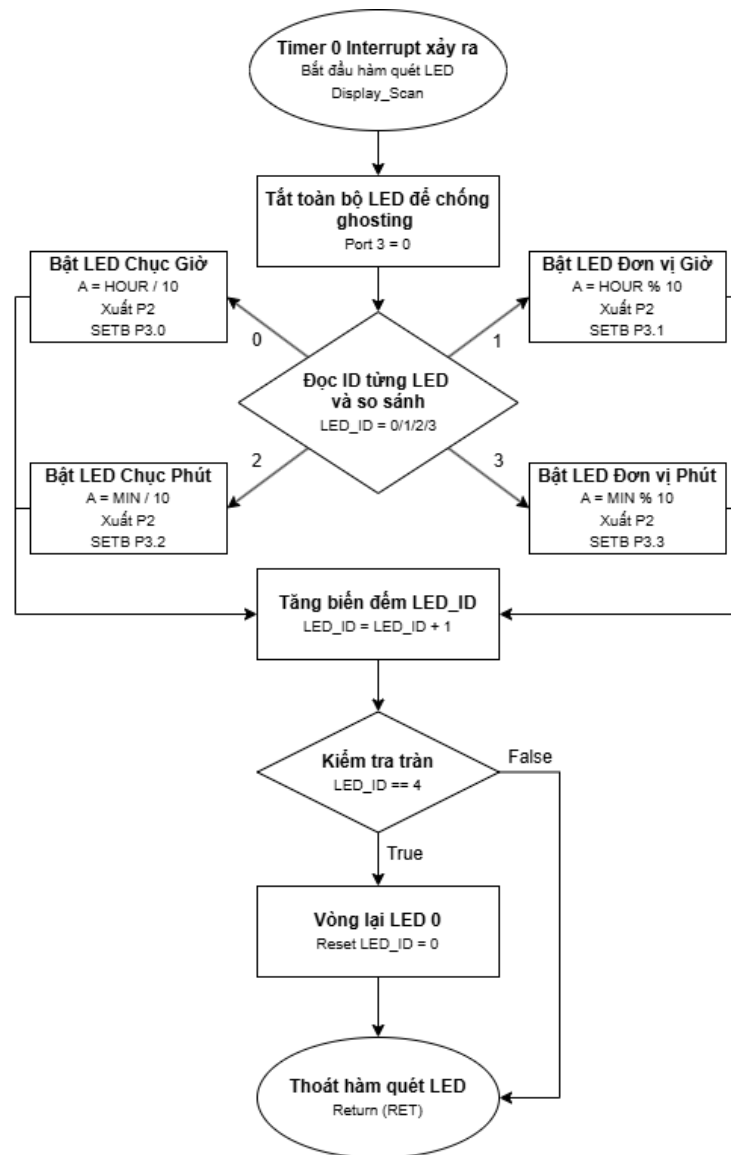
Student code: 24521438

PRACTICE REPORT NO #2:

COMMUNICATION WITH 7-SEGMENT LED AND TIMER/COUNTER

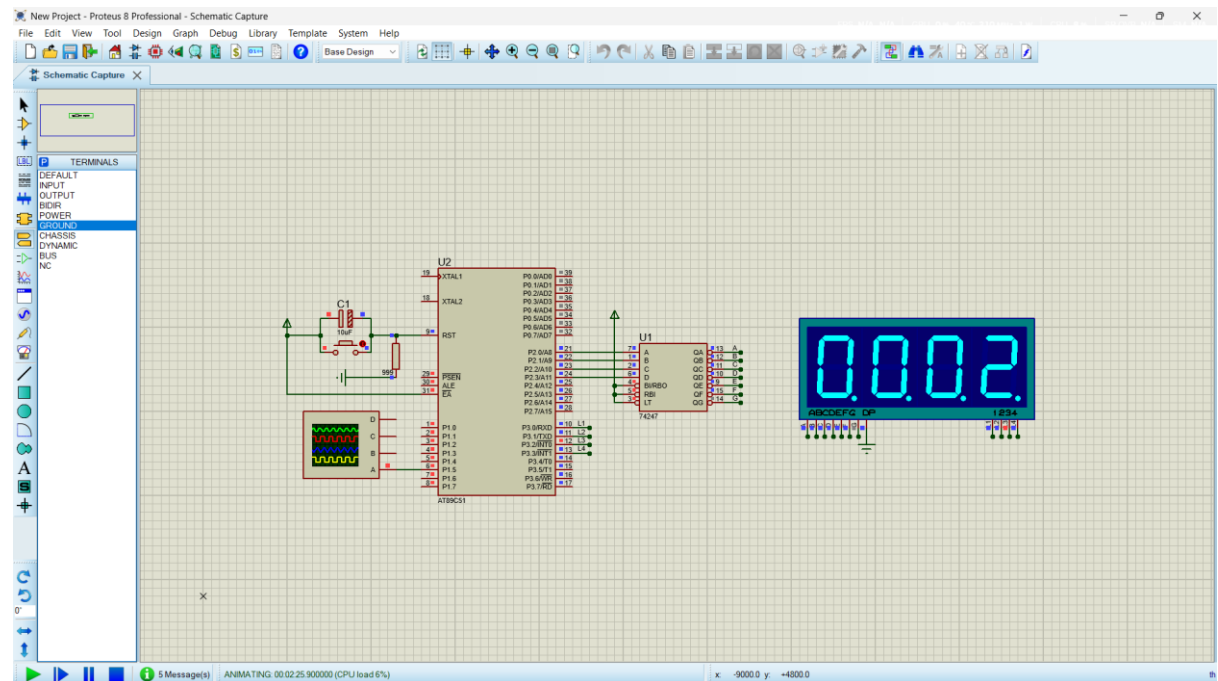
I. Content 1

Present and draw a flowchart of the LED scanning algorithm applied to display a 7-segment LED.

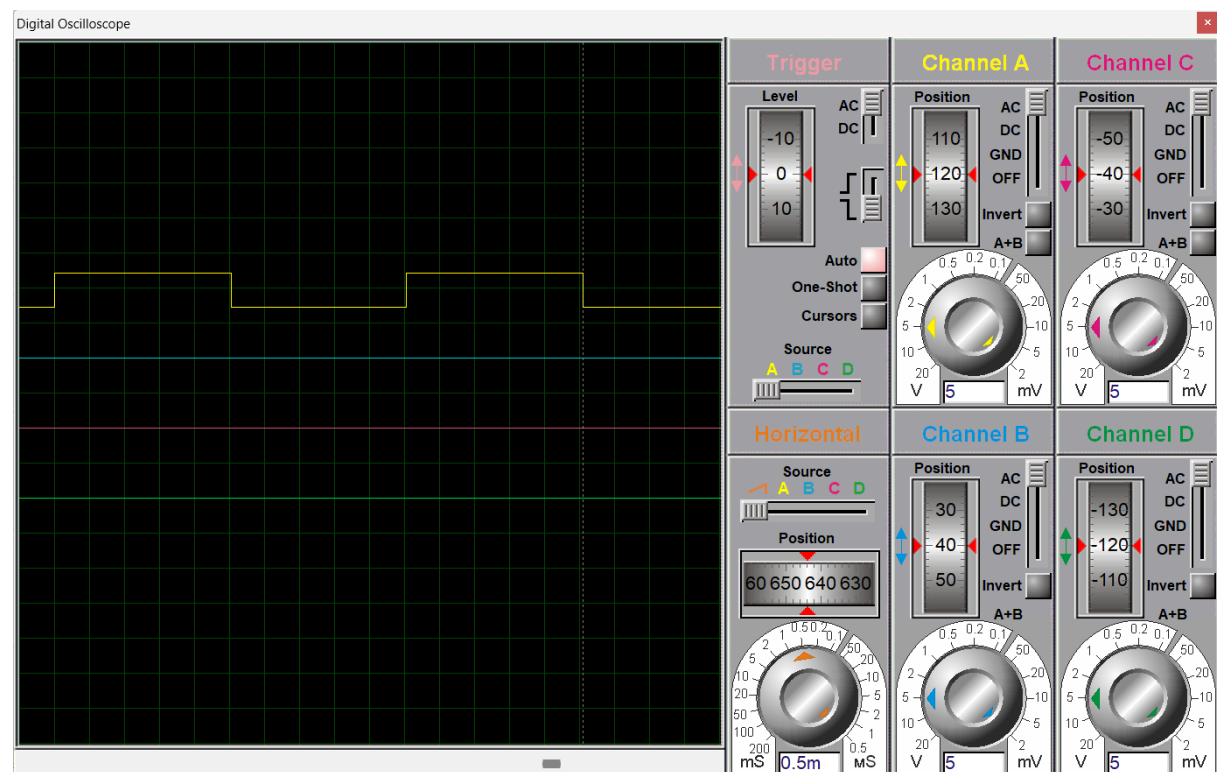


II. Content 2

Using the 8051 microcontroller's Timer, design a clock circuit with 24h format with the initial time set in the source code.



Mạch đồng hồ 24h sử dụng Timer 0 Mode 1



Kiểm chứng chu kỳ quét LED (2.5ms) trên Oscilloscope. Sóng vuông đo tại chân tín hiệu có độ rộng xung bằng chính xác 5 ô (với mức chia thời gian Timebase = 0.5ms/div)

```

=====
;
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
=====

$NOMOD51
$INCLUDE (8051.MCU)

=====
;
; DEFINITIONS
=====

; Crystal 11.0592MHz, 10ms 4 LED (2.5ms ON, 7.5ms OFF per LED)
; Load value: 65536 - (2500 * 11059200 / 12000000) = F700H
LOAD_TH0 EQU 0F7H
LOAD_TL0 EQU 00H
TICK_THRESHOLD EQU 200; 1000 / 2.5 = 400 ticks = 1s -> 0.5s = 200 ticks

INIT_H EQU 00; Initial load hour value
INIT_M EQU 00; Initial load minute value
INIT_S EQU 00; Initial load second value

=====
;
; VARIABLES
=====

MIN DATA 31H ; Current minute value
HOUR DATA 32H; Current hour value
SEC DATA 30H; Current second value

TICK_CNT DATA 33H; Tick count (2.5ms per tick)
HALF_FLAG DATA 34H; Flag for 0.5s status
LED_ID DATA 35H; LED from 0 to 3

=====
;
; RESET and INTERRUPT VECTORS
=====

```

```

; Reset Vector
org 0000h
jmp Start

; Timer 0 Interrupt Vector
org 000Bh
jmp Timer0_ISR

;=====
; CODE SEGMENT
;=====

org 0100h
Start:

; Initialize Stack Pointer
MOV SP, #50H

; Initialize variables
MOV HOUR, #INIT_H
MOV MIN, #INIT_M
MOV SEC, #INIT_S
MOV TICK_CNT, #0
MOV HALF_FLAG, #0
MOV LED_ID, #0

; Initialize Timer 0 value
MOV TMOD, #01H; Timer 0 in mode 1
MOV TH0, #LOAD_TH0
MOV TL0, #LOAD_TL0
SETB ET0; Enable Timer 0 interrupt
SETB EA ; Enable global interrupts
SETB TR0; Start Timer 0

Loop:
jmp Loop

; Timer 0 ISR

```

Timer0_ISR:

CPL P1.5; Toggle P1.5 for oscilloscope measurement

; Reload Timer 0

CLR TR0

MOV TH0, #LOAD_TH0

MOV TL0, #LOAD_TL0

SETB TR0

; Context saving (save to stack)

PUSH ACC

PUSH B

PUSH PSW

; LED Muxing

ACALL Display_Scan; Refresh 1 of 4 LED every 2.5ms

; Calculate time

INC TICK_CNT; Increase tick count every 2.5ms

MOV A, TICK_CNT; Load current tick to A

CJNE A, #TICK_THRESHOLD, ISR_Exit; If tick count != 200, exit ISR

; Reach 0.5s threshold

MOV TICK_CNT, #0; Reset tick count for the next 0.5s

INC HALF_FLAG; Increase half second status flag

MOV A, HALF_FLAG; Load flag to A to check for full second

CJNE A, #2, ISR_Exit; If HALF_FLAG != 2, exit ISR (only 0.5s passed)

; Reach 1s threshold

MOV HALF_FLAG, #0; Reset flag to start a new second count

INC SEC; Increase second value

MOV A, SEC; Move seconds to A for limit checking

CJNE A, #60, ISR_Exit; If seconds != 60, exit ISR. Keep counting seconds

; Reach 1m threshold

MOV SEC, #0

INC MIN

MOV A, MIN

```
CJNE A, #60, ISR_Exit
```

```
; Reach 1h threshold
```

```
MOV MIN, #0
```

```
INC HOUR
```

```
MOV A, HOUR
```

```
CJNE A, #24, ISR_Exit
```

```
; Reach 24h threshold, reset to 0
```

```
MOV HOUR, #0
```

```
ISR_Exit:
```

```
; Context restoring
```

```
POP PSW
```

```
POP B
```

```
POP ACC
```

```
RETI
```

```
Display_Scan: ; Refresh 1 of 4 LED every 2.5ms
```

```
MOV P3, #0; Clear all LED to avoid ghosting
```

```
MOV A, LED_ID; Load current active LED index
```

```
CJNE A, #0, Scan_L1; If index != 0, check next LED
```

```
; LED 0: Hour Tens
```

```
MOV A, HOUR
```

```
MOV B, #10
```

```
DIV AB; A = HOUR / 10 (Quotient), B = HOUR % 10 (Remainder)
```

```
MOV P2, A; Output segments to Port 2
```

```
SETB P3.0; Enable CA for LED 0
```

```
SJMP Next_ID; Jump to index management
```

```
Scan_L1:
```

```
CJNE A, #1, Scan_L2
```

```
; LED 1: Hour Units
```

```
MOV A, HOUR
```

```
MOV B, #10
```

```
DIV AB
```

```
MOV A, B
MOV P2, A
SETB P3.1
SJMP Next_ID
```

Scan_L2:

```
CJNE A, #2, Scan_L3
; LED 2: Minute Tens
MOV A, MIN
MOV B, #10
DIV AB
MOV P2, A
SETB P3.2
SJMP Next_ID
```

Scan_L3:

```
; LED 3: Minute Units
MOV A, MIN
MOV B, #10
DIV AB
MOV A, B
MOV P2, A
SETB P3.3
```

Next_ID:

```
INC LED_ID; Move to next LED for the next interrupt
MOV A, LED_ID
CJNE A, #4, Scan_Done; Check if all 4 LEDs are scanned
MOV LED_ID, #0; Wrap around to first LED
```

Scan_Done:

```
RET
```

```
;=====
END
```

Giải thích chi tiết:

- **Vi điều khiển:** AT89C51 sử dụng thạch anh 11.0592MHz (giúp chia tần số Timer ra số đẹp, không bị sai số).

➤ **Cấu trúc Port:**

- **Port 2 (P2):** Chuyên làm Data Bus, xuất mã BCD 4-bit sang IC giải mã 74247.
- **Port 3 (P3):** Chuyên làm Control Bus, điều khiển cấp nguồn cho 4 LED 7 đoạn (LED_ID từ 0 đến 3).

➤ **Logic điều khiển:**

Port 2: Xuất trực tiếp giá trị số (từ 0 đến 9) ra 4 chân P2.0 - P2.3.

Port 3: Bit = 1 (SETB P3.x): Cấp nguồn VCC → LED sáng. Bit = 0 (MOV P3, #0): Ngắt nguồn → LED tắt.

Phần 1: Định nghĩa hằng số (DEFINITIONS) và cấp phát các biến (VARIABLES)

```
=====
;
; DEFINITIONS
;
=====

; Crystal 11.0592MHz, 10ms 4 LED (2.5ms ON, 7.5ms OFF per LED)
; Load value: 65536 - (2500 * 11059200 / 12000000) = F700H
LOAD_TH0 EQU 0F7H
LOAD_TL0 EQU 00H
TICK_THRESHOLD EQU 200; 1000 / 2.5 = 400 ticks = 1s -> 0.5s = 200 ticks

INIT_H EQU 00; Initial load hour value
INIT_M EQU 00; Initial load minute value
INIT_S EQU 00; Initial load second value

=====
; VARIABLES
;
=====

MIN DATA 31H ; Current minute value
HOUR DATA 32H; Current hour value
SEC DATA 30H; Current second value

TICK_CNT DATA 33H; Tick count (2.5ms per tick)
HALF_FLAG DATA 34H; Flag for 0.5s status
LED_ID DATA 35H; LED from 0 to 3
```

- **EQU (Equate):** Gán tên cho một hằng số. CPU sẽ tự động thay chữ LOAD_TH0 bằng số 0F7H khi biên dịch, tương tự cho LOAD_TL0. Giá trị F700H này được tính toán chuẩn xác từ thạch anh 11.0592MHz để tạo ra độ trễ đúng 2.5ms. ·

- **TICK_THRESHOLD EQU 200:** Định nghĩa ngưỡng đếm nhịp. 1 giây có 400 nhịp 2.5ms, vậy 0.5 giây sẽ cần 200 nhịp.
- **DATA:** Cấp phát các ô nhớ trong vùng RAM nội của 8051 (địa chỉ 30H đến 35H) để chứa dữ liệu cho các biến đếm thời gian thực (Giờ, Phút, Giây) và các cờ trạng thái.

Phần 2: Bảng vector ngắt (RESET and INTERRUPT VECTORS)

```
=====
;
; RESET and INTERRUPT VECTORS
;
=====

; Reset Vector
org 0000h
jmp Start

; Timer 0 Interrupt Vector
org 000Bh
jmp Timer0_ISR
```

- **org 0000h & jmp Start:** Khi mạch vừa được cấp điện hoặc bấm nút Reset, CPU luôn bắt đầu chạy từ địa chỉ 0000H. Lệnh jmp Start chỉ đường cho CPU nhảy qua khu vực Vector ngắt để đi đến phần khởi tạo chương trình chính.
- **org 000Bh & jmp Timer0_ISR:** 000BH là địa chỉ của ngắt Timer 0. Mỗi khi Timer đếm tràn (sau 2.5ms), CPU sẽ lập tức quay về đây. Lệnh jmp Timer0_ISR sẽ điều hướng CPU chạy thẳng vào hàm xử lý ngắt.

Phần 3: Khởi tạo hệ thống và vòng lặp chính (Main Loop)

```
=====
;
; CODE SEGMENT
;
=====

org 0100h
Start:
; Initialize Stack Pointer
MOV SP, #50H

; Initialize variables
MOV HOUR, #INIT_H
```

```

MOV MIN, #INIT_M
MOV SEC, #INIT_S
MOV TICK_CNT, #0
MOV HALF_FLAG, #0
MOV LED_ID, #0

; Initialize Timer 0 value
MOV TMOD, #01H; Timer 0 in mode 1
MOV TH0, #LOAD_TH0
MOV TL0, #LOAD_TL0
SETB ET0; Enable Timer 0 interrupt
SETB EA ; Enable global interrupts
SETB TR0; Start Timer 0

```

Loop:

```

    jmp Loop

```

- **MOV SP, #50H:** Đẩy con trỏ Stack (SP) lên vùng RAM trống 50H để không ghi đè vào khu vực RAM 30H-35H đang dùng chứa biến dữ liệu.
- **MOV TMOD, #01H:** Cấu hình Timer 0 hoạt động ở Chế độ 1 (Bộ đếm 16-bit).
- **SETB ET0 / EA / TR0:** Kích hoạt ngắt Timer 0, mở khóa ngắt toàn cục và bắt đầu cho Timer 0 chạy.
- **jmp Loop:** Vòng lặp vô tận (Idle loop). CPU sẽ ở đây nghỉ ngơi, hoàn toàn không sử dụng hàm delay gây kẹt hệ thống.

Phần 4: Trình phục vụ ngắt Timer 0 (Timer0_ISR)

```

; Timer 0 ISR
Timer0_ISR:
    CPL P1.5; Toggle P1.5 for oscilloscope measurement

; Reload Timer 0
CLR TR0
MOV TH0, #LOAD_TH0
MOV TL0, #LOAD_TL0
SETB TR0

; Context saving (save to stack)
PUSH ACC

```

```

PUSH B
PUSH PSW

; LED Muxing
ACALL Display_Scan

; Calculate time
INC TICK_CNT; Increase tick count every 2.5ms
MOV A, TICK_CNT; Load current tick to A
CJNE A, #TICK_THRESHOLD, ISR_Exit; If tick count != 200, exit ISR

; Reach 0.5s threshold
MOV TICK_CNT, #0; Reset tick count for the next 0.5s
INC HALF_FLAG; Increase half second status flag
MOV A, HALF_FLAG; Load flag to A to check for full second
CJNE A, #2, ISR_Exit; If HALF_FLAG != 2, exit ISR (only 0.5s passed)

; Reach 1s threshold
MOV HALF_FLAG, #0; Reset flag to start a new second count
INC SEC; Increase second value
MOV A, SEC; Move seconds to A for limit checking
CJNE A, #60, ISR_Exit; If seconds != 60, exit ISR. Keep counting seconds

; Reach 1m threshold
MOV SEC, #0
INC MIN
MOV A, MIN
CJNE A, #60, ISR_Exit

; Reach 1h threshold
MOV MIN, #0
INC HOUR
MOV A, HOUR
CJNE A, #24, ISR_Exit

; Reach 24h threshold, reset to 0
MOV HOUR, #0

```

ISR_Exit:

```
; Context restoring
POP PSW
POP B
POP ACC
RETI
```

- **CPL P1.5:** Đảo mức logic tại chân P1.5, tạo xung vuông để kiểm chứng chu kỳ 2.5ms trên Oscilloscope.

- **CLR TR0 / Nạp TH0-TL0 / SETB TR0:** Nạp lại giá trị F700H ngay lập tức để duy trì chu kỳ 2.5ms chuẩn xác.

- **PUSH / POP:** Lưu (PUSH) các thanh ghi A, B, và PSW vào ngăn xếp (stack) trước khi xử lý ngắt, và lấy ra (POP) theo đúng thứ tự ngược lại trước khi thoát để bảo toàn dữ liệu chương trình chính.

- **ACALL Display_Scan:** Gọi chương trình con quét LED.

- **Logic thời gian (INC và CJNE):** Mỗi lần ngắt (2.5ms) sẽ tăng biến TICK_CNT. Cứ đủ ngưỡng (TICK_THRESHOLD = 200) thì tạo ra nửa giây. Cứ đủ 2 nửa giây thì tăng SEC (giây). Đạt ngưỡng 60 sẽ tăng MIN, rồi tăng HOUR.

- **RETI:** Lệnh kết thúc ISR, xóa cờ báo bận trong phần cứng để sẵn sàng nhận lần ngắt sau.

Phần 5: Chương trình con quét LED (Display_Scan)

Display_Scan: ; Refresh 1 of 4 LED every 2.5ms

```
MOV P3, #0; Clear all LED to avoid ghosting
```

```
MOV A, LED_ID; Load current active LED index
```

```
CJNE A, #0, Scan_L1; If index != 0, check next LED
```

```
; LED 0: Hour Tens
```

```
MOV A, HOUR
```

```
MOV B, #10
```

```
DIV AB; A = HOUR / 10 (Quotient), B = HOUR % 10 (Remainder)
```

```
MOV P2, A; Output segments to Port 2
```

```
SETB P3.0; Enable CA for LED 0
```

```
SJMP Next_ID; Jump to index management
```

Scan_L1:

```
CJNE A, #1, Scan_L2
```

```
; LED 1: Hour Units
```

```
MOV A, HOUR
MOV B, #10
DIV AB
MOV A, B
MOV P2, A
SETB P3.1
SJMP Next_ID
```

Scan_L2:

```
CJNE A, #2, Scan_L3
; LED 2: Minute Tens
MOV A, MIN
MOV B, #10
DIV AB
MOV P2, A
SETB P3.2
SJMP Next_ID
```

Scan_L3:

```
; LED 3: Minute Units
MOV A, MIN
MOV B, #10
DIV AB
MOV A, B
MOV P2, A
SETB P3.3
```

Next_ID:

```
INC LED_ID; Move to next LED for the next interrupt
MOV A, LED_ID
CJNE A, #4, Scan_Done; Check if all 4 LEDs are scanned
MOV LED_ID, #0; Wrap around to first LED
```

Scan_Done:

```
RET
```

```
;=====
END
```

- **MOV P3, #0:** Ngắt nguồn toàn bộ LED trước khi xuất dữ liệu mới. Đây là bước anti-ghosting bắt buộc trong kỹ thuật quét LED.
- **DIV AB:** Tách số. Lấy giá trị thời gian chia 10. Phần nguyên vào A (Chữ số hàng chục), phần dư vào B (Chữ số hàng đơn vị).
- **MOV P2, A & SETB P3.x:** Xuất dữ liệu BCD (từ 0-9) ra Port 2 để IC 74247 giải mã. Kích mức 1 vào chân Port 3 tương ứng để cấp nguồn, làm LED sáng lên.
- **INC LED_ID & Ép về 0:** Tăng chỉ số LED để chuẩn bị hiển thị số tiếp theo ở chu kỳ sau. Khi ID chạm mức 4 (vượt quá số lượng 4 LED thực tế), ép nó quay về 0, tạo thành vòng quét tuần hoàn vô tận (0-1-2-3-0...) đánh lừa thị giác người xem, tạo cảm giác 4 LED sáng đồng thời.

III. Exercises

1. With the above clock design, use a loop to create a delay instead of Timer. State the advantages and disadvantages of the two methods.

```

=====
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
=====

$NOMOD51
$INCLUDE (8051.MCU)

=====
; DEFINITIONS
=====

; Crystal 11.0592MHz, 10ms 4 LED (2.5ms ON, 7.5ms OFF per LED)
; Software delay: 2500 * 11059200 / 12000000 = 2304 MC
TICK_THRESHOLD EQU 200; 1000 / 2.5 = 400 ticks = 1s -> 0.5s = 200 ticks

INIT_H EQU 00; Initial load hour value
INIT_M EQU 00; Initial load minute value
INIT_S EQU 00; Initial load second value

=====
; VARIABLES

```

```

;=====

MIN DATA 31H ; Current minute value
HOUR DATA 32H; Current hour value
SEC DATA 30H; Current second value

TICK_CNT DATA 33H; Tick count (2.5ms per tick)
HALF_FLAG DATA 34H; Flag for 0.5s status
LED_ID DATA 35H; LED from 0 to 3

;=====
; RESET and INTERRUPT VECTORS
;=====

; Reset Vector
org 0000h
jmp Start

;=====
; CODE SEGMENT
;=====

org 0100h
Start:
; Initialize Stack Pointer
MOV SP, #50H

; Initialize variables
MOV HOUR, #INIT_H
MOV MIN, #INIT_M
MOV SEC, #INIT_S
MOV TICK_CNT, #0
MOV HALF_FLAG, #0
MOV LED_ID, #0

Loop:
CPL P1.5; Toggle P1.5 for oscilloscope measurement
ACALL Delay_Tick; Create 2.5ms delay per tick (software delay)

```

ACALL Display_Scan; Refresh 1 of 4 LED every 2.5ms

ACALL Update_Clock; Update clock every 2.5ms

SJMP Loop

Update_Clock:

; Calculate time from one 2.5ms software tick

INC TICK_CNT; Increase tick count every 2.5ms

MOV A, TICK_CNT; Load current tick to A

CJNE A, #TICK_THRESHOLD, Update_Done; If tick count != 200, exit

; Reach 0.5s threshold

MOV TICK_CNT, #0; Reset tick count for the next 0.5s

INC HALF_FLAG; Increase half second status flag

MOV A, HALF_FLAG; Load flag to A to check for full second

CJNE A, #2, Update_Done; If HALF_FLAG != 2, only 0.5s passed

; Reach 1s threshold

MOV HALF_FLAG, #0; Reset flag to start a new second count

INC SEC; Increase second value

MOV A, SEC; Move seconds to A for limit checking

CJNE A, #60, Update_Done; If seconds != 60, keep counting seconds

; Reach 1m threshold

MOV SEC, #0

INC MIN

MOV A, MIN

CJNE A, #60, Update_Done

; Reach 1h threshold

MOV MIN, #0

INC HOUR

MOV A, HOUR

CJNE A, #24, Update_Done

; Reach 24h threshold, reset to 0

MOV HOUR, #0

Update_Done:

RET

Delay_Tick::; 2MC - ACALL overhead

MOV R6, #6; 1MC - Initialize 6 outer loop

Delay_L1:

MOV R7, #190; 6MC - Initialize 190 inner loop

Delay_L2:

DJNZ R7, Delay_L2; 2280MC - $6 * 190 * 2$ MC (2MC per DJNZ)

DJNZ R6, Delay_L1; 12MC - $6 * 2$ MC (2MC per DJNZ)

NOP; 1MC - For 2304 MC total

RET; 2MC - RET overhead

Display_Scan:

MOV P3, #0; Clear all LED to avoid ghosting

MOV A, LED_ID; Load current active LED index

CJNE A, #0, Scan_L1; If index != 0, check next LED

; LED 0: Hour Tens

MOV A, HOUR

MOV B, #10

DIV AB; A = HOUR / 10 (Quotient), B = HOUR % 10 (Remainder)

MOV P2, A; Output segments to Port 2

SETB P3.0; Enable CA for LED 0

SJMP Next_ID; Jump to index management

Scan_L1:

CJNE A, #1, Scan_L2

; LED 1: Hour Units

MOV A, HOUR

MOV B, #10

DIV AB

MOV A, B

MOV P2, A

SETB P3.1

SJMP Next_ID

Scan_L2:

```

CJNE A, #2, Scan_L3
; LED 2: Minute Tens
MOV A, MIN
MOV B, #10
DIV AB
MOV P2, A
SETB P3.2
SJMP Next_ID

Scan_L3:
; LED 3: Minute Units
MOV A, MIN
MOV B, #10
DIV AB
MOV A, B
MOV P2, A
SETB P3.3

Next_ID:
INC LED_ID; Move to next LED for the next interrupt
MOV A, LED_ID
CJNE A, #4, Scan_Done; Check if all 4 LEDs are scanned
MOV LED_ID, #0; Wrap around to first LED

Scan_Done:
RET

=====
END

```

Giải thích chi tiết:

- **Phương pháp Software Delay (Tạo trễ bằng phần mềm):** Khác với việc dùng Timer phần cứng, CPU tự tạo ra thời gian trễ bằng cách chạy các vòng lặp vô nghĩa (DJNZ) để tiêu tốn chính xác số lượng chu kỳ máy (Machine Cycles - MC) cần thiết.
- **Phương pháp định thời:** Với thạch anh 11.0592MHz, 1 MC tốn khoảng 1.085 μ s. Để có được 2.5ms, hệ thống cần tiêu tốn 2304 MC. Hàm Delay_Tick được thiết kế chặt chẽ qua phép tính: $2 \text{ (ACALL)} + 1 + 6 + 2280 + 12 + 1 \text{ (NOP)} + 2 \text{ (RET)} = 2304 \text{ MC}$.

```
Delay_Tick;; 2MC - ACALL overhead
    MOV R6, #6; 1MC - Initialize 6 outer loop
Delay_L1:
    MOV R7, #190; 6MC - Initialize 190 inner loop
Delay_L2:
    DJNZ R7, Delay_L2; 2280MC - 6 * 190 * 2 MC (2MC per DJNZ)
    DJNZ R6, Delay_L1; 12MC - 6 * 2 MC (2MC per DJNZ)
    NOP; 1MC - For 2304 MC total
    RET; 2MC - RET overhead
```

- **Kiến trúc vòng lặp tuần tự (Polling Loop):** Vòng lặp Loop hoạt động liên tục, gọi các hàm theo thứ tự nghiêm ngặt: Tạo trễ 2.5ms (Delay_Tick) $\rightarrow$ Quét hiển thị 1 LED (Display_Scan) $\rightarrow$ Tính toán thời gian (Update_Clock).
- **Đo đạc thực tế:** Lệnh CPL P1.5 được đặt ngay đầu vòng lặp chính nhằm tạo ra một xung vuông. Khi cắm Oscilloscope vào chân này, ta sẽ đo được chu kỳ làm việc thực tế của CPU để xác minh thuật toán tạo trễ.

Các phần đã chỉnh sửa so với kiến trúc Hardware Timer:

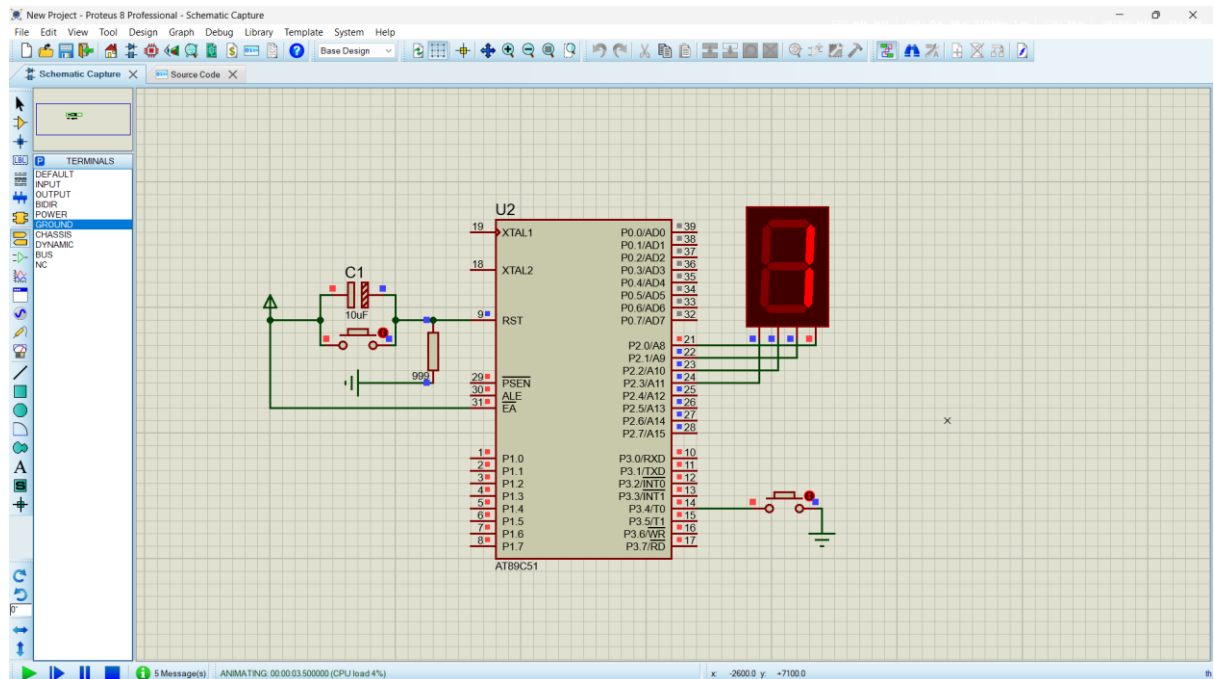
1. **Xóa bỏ phần cứng Timer:** Loại bỏ hoàn toàn bảng Vector ngắt (org 000Bh), các bước cấu hình thanh ghi Timer (TMOD, TH0, TL0) và các cờ ngắt (EA, ET0, TR0).
2. **Xóa bỏ lệnh quản lý Ngăn xếp:** Gỡ bỏ các lệnh PUSH, POP và RETI vì hệ thống không còn sử dụng ISR, không lo bị ngắt ngang luồng dữ liệu chính.
3. **Thêm mới khối Delay_Tick:** Sử dụng 2 thanh ghi R6, R7 kết hợp lệnh DJNZ và lệnh đệm NOP để khóa (block) CPU đứng yên trong đúng 2.5ms.
4. **Cấu trúc lại luồng xử lý:** Mang toàn bộ logic cộng dồn thời gian (từ Timer0_ISR cũ) ra ngoài thành chương trình con Update_Clock, và đưa lệnh đo xung CPL P1.5 vào vòng lặp Loop chính.

Ưu và nhược điểm của 2 phương pháp:

Tiêu chí	Phương pháp Hardware Timer (Dùng Ngắt)	Phương pháp Software Delay (Vòng lặp)
Bản chất hoạt động	Sử dụng bộ đếm độc lập tích hợp sẵn trong vi điều khiển để tự động đếm nhịp thạch anh.	Ép CPU chạy các vòng lặp vô nghĩa (DJNZ) để tiêu tốn chính xác số chu kỳ máy cần thiết.
Độ chính xác	Rất cao. Nhịp đếm được chốt cứng bởi thạch anh. Chu kỳ đáp	Thấp hơn. Dễ sinh ra sai số (Jitter) do tổng thời gian bị

	ứng ổn định, không bị ảnh hưởng bởi độ dài của mã nguồn.	cộng dồn với các hàm xử lý khác (như hàm quét LED, cộng thời gian).
Hiệu suất CPU	Tối ưu (Đa nhiệm). Trong lúc Timer tự đếm 2.5ms, CPU hoàn toàn rảnh rỗi để làm việc khác hoặc chuyển sang chế độ ngủ (Idle) tiết kiệm pin.	Lãng phí (Blocking). CPU bị giam hoàn toàn trong vòng lặp Delay, không thể làm thêm bất cứ việc gì khác trong lúc chờ.
Độ phức tạp	Phức tạp. Phải tính toán nạp thanh ghi (TH/TL), cấu hình mode (TMOD), bật cờ ngắt (EA, ET) và quản lý stack (PUSH/POP).	Đơn giản. Chỉ cần dùng vài lệnh vòng lặp cơ bản. Dễ hiểu, dễ tính toán số chu kỳ máy trực tiếp trên code, luồng chạy tuần tự dễ theo dõi.
Khả năng mở rộng	Rất dễ. Có thể thêm nhiều chức năng vào chương trình chính mà không sợ làm lệch nhịp đồng hồ.	Rất khó. Thêm bất kỳ một dòng lệnh nào vào vòng lặp chính cũng sẽ làm chu kỳ quét LED bị dẫn ra và chạy sai giờ.

- Design a system to count the number of button presses using Counter of the AT89C51. The counted value is displayed on a 7-segment LED display.



```

=====
;
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
=====

$NOMOD51
$INCLUDE (8051.MCU)

=====
;
; DEFINITIONS
=====

=====
;
; VARIABLES
=====

COUNT DATA 30H ; Counter variable (0-9)

=====
;
; RESET and INTERRUPT VECTORS
=====

; Reset Vector
org 0000h
jmp Start

=====
;
; CODE SEGMENT
=====

org 0100h
Start:
; Initialize Stack Pointer
MOV SP, #50H

; Initialize counter variable to 0
MOV COUNT, #0

```

```

; Initialize Counter 0 in Mode 2 (8-bit Auto-reload)
MOV TMOD, #06H; TMOD = 0000 0110B (C/T=1, Mode=2)
MOV TH0, #0FFH; Reload value = 255 (0FFH)
MOV TL0, #0FFH; Initial count value
SETB TR0; Start Counter 0 to monitor P3.4

Loop:
; Check TF0 flag for overflow (indicates a button press)
JNB TF0, Display; If TF0 is 0, skip increment and keep display

; Handle button press event
CLR TF0; Clear overflow flag for the next press
INC COUNT; Increment the counter variable

; Limit 0-9 for display
MOV A, COUNT
CJNE A, #10, Display; If COUNT != 10, jump to Display
MOV COUNT, #0; Reset counter variable to 0 if it reaches 10

Display:
MOV P2, COUNT; Output the counter variable to Port 2
SJMP Loop

;=====
END

```

Giải thích chi tiết:

Chương trình hoạt động dựa trên phương pháp Polling (Hỏi vòng) để kiểm tra trạng thái của Counter 0. Quy trình thực hiện như sau:

1. **Khởi tạo:** Thiết lập stack pointer, xóa biến đếm về 0.
2. **Cấu hình phần cứng:** Thiết lập Counter 0 ở chế độ 8-bit tự động nạp lại (Mode 2) để đếm xung ngoại tại chân P3.4.
3. **Vòng lặp chính:**
 - Kiểm tra cờ báo tràn TF0.
 - Nếu chưa nhấn nút: Tiếp tục hiển thị giá trị hiện tại ra Port 2.
 - Nếu đã nhấn nút: Xóa cờ TF0, tăng biến đếm COUNT.
4. **Kiểm soát dữ liệu:** So sánh biến COUNT với 10. Nếu bằng 10, thực hiện reset biến về 0 để duy trì khoảng đếm 0-9 và tiếp tục hiển thị.
5. **Hiển thị:** Xuất giá trị biến COUNT ra Port 2 để LED 7 đoạn (BCD) cập nhật hình ảnh.

A. Khai báo và Khởi tạo

- COUNT DATA 30H: Định nghĩa địa chỉ lưu trữ biến đếm trong RAM nội.
- MOV SP, #50H: Khởi tạo con trỏ SP.
- MOV COUNT, #0: Thiết lập giá trị đếm ban đầu bằng 0.

B. Thiết lập Counter 0 (Mode 2)

- MOV TMOD, #06H: Cấu hình thanh ghi TMOD. Bit C/T=1 chọn chế độ đếm xung ngoài (Counter). M1=1, M0=0 chọn Mode 2 (8-bit Auto-reload).
- MOV TH0, #0FFH: Nạp giá trị nạp lại (Reload value). Giá trị 255 giúp bộ đếm tràn ngay khi nhận 1 xung ngoài duy nhất.
- MOV TL0, #0FFH: Nạp giá trị khởi tạo cho bộ đếm.
- SETB TR0: Cho phép bộ đếm 0 bắt đầu hoạt động.

C. Xử lý logic và Hiển thị

- JNB TF0, Display: Kiểm tra cờ tràn TF0. Nếu cờ bằng 0 (chưa có xung), nhảy đến nhãn hiển thị (Display)
- CLR TF0: Xóa cờ tràn bằng phần mềm để chuẩn bị cho chu kỳ đếm tiếp theo.
- INC COUNT: Tăng giá trị biến đếm khi phát hiện sự kiện nhấn nút.
- CJNE A, #10, Display: So sánh giá trị trong thanh ghi A với 10. Nếu chưa đạt 10, bỏ qua lệnh Reset để tiếp tục hiển thị.
- MOV P2, COUNT: Xuất dữ liệu nhị phân từ biến đếm ra Port 2 để điều khiển LED 7 đoạn loại giải mã BCD.

IV. References

- TS. Vũ Đức Lung, TS. Lê Quang Minh, ThS. Phan Đình Duy, *Giáo trình Vi điều khiển*, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2015
- Link video 1 (Content 2):
<https://drive.google.com/file/d/1HPVU9--PUE9193mNrAncxuT4EChPKPko/view?usp=sharing>
- Link video 2 (Exercise 2):
https://drive.google.com/file/d/16VB7nR_SEjWx87pqxoWYnaDVlu0ntiDa/view?usp=sharing